

# 封装

## 封装

```
#include<iostream>
using namespace std;

const double pi=3.1415926;

class circle{
public://访问权限

    //属性 / 成员属性 / 成员变量
    int r;

    //行为 / 成员函数 / 成员方法
    double calculatezc()
    {
        return 2*pi*r;
    }
};//属性和行为都称为成员

int main()
{
    circle c;
    c.r=10;
    cout<<c.calculatezc()<<endl;
    return 0;
}
```

## 封装访问权限

```
#include<iostream>
using namespace std;
//struct默认访问权限是公共public
//class默认访问权限是私有 private

class A1{
    int m_a;//默认为private
};

struct A2{
    int m_a;//默认为public
};

class person{
public://类内可以访问，类外也可以访问
    string m_name;

protected://类内可以访问，类外不可以访问，但儿子可以访问父亲中的保护内容
    string m_car;
}
```

```

private://类内可以访问，内外不可以访问，但儿子不可以访问父亲中的私有内容
    int m_password;

public:
    void func()
    {
        cout<<m_name<<endl;
        cout<<m_car<<endl;
    }
};

int main()
{

    person p;
    p.m_name="zq";
    p.func();

    //A1 a1;
    //a1.m_a=100;A1类默认为private权限，类外禁止访问

    struct A2 a2;
    a2.m_a=100;

    return 0;
}

```

## 成员属性设为私有

```

#include<iostream>
using namespace std;
//struct默认访问权限是公共public
//class默认访问权限是私有 private

class A1{
    int m_a;//默认为private
};

struct A2{
    int m_a;//默认为public
};

class person{
public://类内可以访问，类外也可以访问
    string m_name;

protected://类内可以访问，类外不可以访问，但儿子可以访问父亲中的保护内容
    string m_car;

private://类内可以访问，内外不可以访问，但儿子不可以访问父亲中的私有内容
    int m_password;
}

```

```

public:
    void func()
    {
        cout<<m_name<<endl;
        cout<<m_car<<endl;
    }
};

int main()
{

    person p;
    p.m_name="zq";
    p.func();

    //A1 a1;
    //a1.m_a=100;A1类默认为private权限，类外禁止访问

    struct A2 a2;
    a2.m_a=100;

    return 0;
}

```

## 案例一-立方体类

```

#include<iostream>
using namespace std;

//cube类
class cube{
public:
    void setlength(int l)
    {
        m_l=l;
    }
    int getlength()
    {
        return m_l;
    }

    void setwidth(int w)
    {
        m_w=w;
    }
    int getwidth()
    {
        return m_w;
    }

    void sethigh(int h)
    {
        m_h=h;
    }
}

```

```

    }
    int gethigh()
    {
        return m_h;
    }

    int calculates()
    {
        return 2*m_l*m_w + 2*m_l*m_h + 2*m_w*m_h;
    }

    int calculatev()
    {
        return m_l*m_w*m_h;
    }

    bool samebyclass(cube c)//只需要传一个参数就可以比较判断
    {
        if(calculates()==c.calculates()&&calculatev()==c.calculatev())
        {
            return true;
        }
        else
        {
            return false;
        }
    }

private://属性一般放在private中
    int m_l;
    int m_w;
    int m_h;
};

//same全局函数
bool same(cube c1,cube c2)//bool类型返回值为true/false
{
    if(c1.calculates()==c2.calculates()&&c1.calculatev()==c2.calculatev())
    {
        return true;
    }
    else
    {
        return false;
    }
}

//主函数
int main()
{
    cube c1;
    c1.setlength(10);
    c1.setwidth(20);
    c1.sethigh(30);

```

```

cube c2;
c2.setlength(15);
c2.setwidth(25);
c2.sethigh(5);

bool ret1=same(c1,c2);
if(ret1==true)
{
    cout<<"true"<<endl;
}
else
{
    cout<<"false"<<endl;
}

bool ret2=c1.samebyclass(c2);
if(ret2==true)
{
    cout<<"true"<<endl;
}
else
{
    cout<<"false"<<endl;
}

cout<<c1.calculates()<<endl;
cout<<c1.calculateV()<<endl;

cout<<c2.calculates()<<endl;
cout<<c2.calculateV()<<endl;

return 0;
}

```

## 案例二-点和圆类

```

#include<iostream>
using namespace std;

class point//定义point类（点）
{
public:
    void setx(int x)
    {
        m_x=x;
    }
    int getx()
    {
        return m_x;
    }
}

```

```

    void sety(int y)
    {
        m_y=y;
    }
    int gety()
    {
        return m_y;
    }

private:
    int m_x;
    int m_y;
};

class circle{//定义circle类（圆类）

public:
    void setr(int r)
    {
        m_r=r;
    }
    int getr()
    {
        return m_r;
    }

    void setcenter(point center)
    {
        m_center=center;
    }

    point getcenter()
    {
        return m_center;
    }

private:
    int m_r;//定义半径
    point m_center;//定义圆心
};

void isincircle(circle c,point p)
{
    int distance=(c.getcenter().getx()-p.getx())*(c.getcenter().getx()-p.getx())
+ (c.getcenter().gety()-p.gety())*(c.getcenter().gety()-p.gety());
    int rdistance=c.getr()*c.getr();

    if(distance>rdistance)
    {
        cout<<"in"<<endl;
    }
    else if(distance==rdistance)
    {
        cout<<"on"<<endl;
    }
}

```

```
        else if(distance>rdistance)
        {
            cout<<"out"<<endl;
        }
    }

int main()
{
    circle c;
    point center;

    c.setr(10);
    center.setx(10);
    center.sety(0);
    c.setcenter(center);

    point p;
    p.setx(10);
    p.sety(10);

    isincircle(c,p);
    return 0;
}
```